

Technology Stack

Date	5 July 2026
Team ID	xxxxxx
Project Name	Smart Lender
Maximum Marks	2 Marks

Define Your Technology Stack:

The technology stack defines the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs used to build the solution. A well-chosen technology stack ensures the application is scalable, maintainable, and aligned with project requirements.

Technology Stack Details

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side	HTML5, CSS3, Jinja2 Templates	Pure HTML and CSS were chosen to keep the frontend lightweight and framework-free. Jinja2 (Flask's built-in templating engine) dynamically renders the result page with prediction output and applicant summary. CSS Grid is used for responsive form layouts across desktop and mobile. Google Fonts (Inter) is loaded via CDN for clean, professional typography. No JavaScript framework was needed since the app has a simple two-page flow.
2	Backend / Server-Side	Python 3.x, Flask 3.0.3	Python is the natural choice for an ML-integrated web app due to its rich ecosystem of data science libraries. Flask is a lightweight micro-framework that handles HTTP routing (GET / and POST /predict) with minimal boilerplate. It integrates seamlessly with NumPy and the pickled XGBoost model. Flask also provides built-in Jinja2 templating, making server-side rendering straightforward without needing a separate frontend build step.
3	Database / Data Storage	CSV File (loan.csv), Pickle (.pkl)	The project uses a structured CSV dataset (dataset/loan.csv) for model training — no live transactional database is required since predictions are stateless. The trained XGBoost model is serialized and stored as model.pkl using Python's Pickle library, enabling fast model loading at server startup without retraining. This file-based approach is appropriate for an educational ML project where no user data needs to be persisted.

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
4	Cloud / Hosting / Deployment	Local / Localhost (Flask Dev Server); Recommended: Render / Heroku for production	Currently runs locally on http://0.0.0.0:5000 using Flask's built-in development server. For production deployment, platforms like Render or Heroku are recommended — they support Python/Flask apps, offer free tiers, and require no infrastructure management. The app's stateless design (no DB writes) makes it easy to deploy on any cloud platform supporting Python WSGI apps.
5	Version Control & CI/CD	Git & GitHub	Git is used for source version control to track changes across app.py, train_model.py, templates, and the dataset. GitHub provides remote repository hosting for collaboration and backup. GitHub also enables future CI/CD integration (e.g., GitHub Actions) to automate model retraining or deployment when code is pushed. The project structure is simple enough that manual deployment suffices for the current phase.
6	Third-Party APIs / Other Tools	Google Fonts API; Scikit-learn, XGBoost, Pandas, NumPy, Matplotlib, Seaborn, Jupyter Notebook	Google Fonts API — loads the "Inter" typeface via CDN for consistent UI typography without bundling font files. Scikit-learn — provides Decision Tree, Random Forest, and KNN classifiers, plus LabelEncoder, train_test_split, and accuracy_score for model evaluation. XGBoost — the final chosen model (81.1% test accuracy) for loan prediction. Pandas & NumPy — used for dataset loading, missing value imputation, and feature array construction. Matplotlib & Seaborn — used in the Jupyter notebook for EDA and visualization. Jupyter Notebook — used for exploratory data analysis and prototyping before building the Flask app.